SWC3

Software-Entwicklung mit C und C++

Erik Pitzer

WS 2026/27

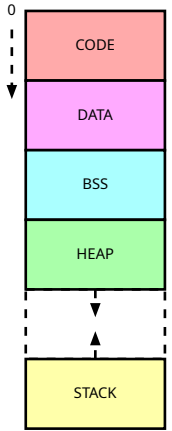
FH OÖ - Campus Hagenberg - Software Engineering



5. C Speicherverwaltung

Überblick Speichersegmente

- Programm hat zur Laufzeit Speicher in Segmenten
  - wird von Loader aus Objektdatei[1] geladen
- Code- oder Text-Segment
  - ausführbarer Maschinencode[2]
- Data-Segment
  - Konstanten & initialisierte globale Variablen
- BSS[3]-Segment (ebenfalls Daten)
  - null-initialisierte globale Variablen
- Heap-Segment (Halde)
  - “manuell” verwalteter, dynamischer Speicher
- Stack-Segment
  - automatisch verwalteter Speicher
  - “Stackframes” für Prozeduraufrufe
    - Rücksprungadresse
    - Prozedurparameter
    - lokale Variablen (`auto`)



[1] Objektdatei wird daher auch als *memory image* Bezeichnet
[2] oft durch Betriebssystem vor Veränderung geschützt
[3] BSS Steht für *“Block Started by Symbol”* da kein Speicher in der Objektdatei reserviert werden muss

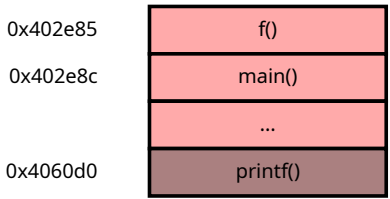
Code Segment

- enthält Maschinensprache
- kann direkt ausgeführt werden
- Funktionsname ist Zeiger ins Code-Segment

```c
int f() { }

int main() {
  SHOW_TYPE(f);
  SHOW_TYPE(main);
  SHOW_TYPE(printf);
}
```

```
f = 0x402e85 (function)
main = 0x402e8c (function)
printf = 0x4060d0 (library function)
```



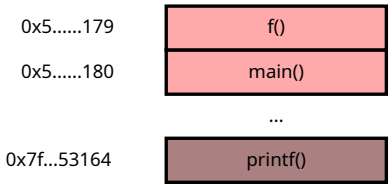
Code Segmente von gemeinsamen Bibliotheken

- Bibliotheksfunktionen oft gemeinsam mit anderen Programmen genutzt (shared library)
- Zeiger von `printf()` daher in anderes Segment

```c
int f() { }

int main() {
  SHOW_TYPE(f);
  SHOW_TYPE(main);
  SHOW_TYPE(printf);
}
```

```
f = 0x557b4a1ec179 (function)
main = 0x557b4a1ec180 (function)
printf = 0x7f8c30e56a64 (library function)
```

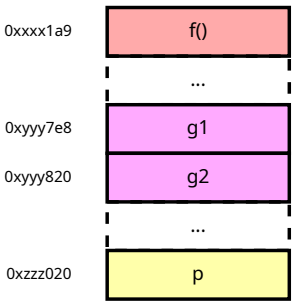


Daten-Segment

- globale Variablen und lokale `static` Variablen sind im Datensegment
- oft sehr nahe bei Code-Segment
- teilweise sogar Schreibgeschützt für `const`-Variablen

```c
static int g1;
void f(int p) {
  static int g2;
  SHOW(&g1); SHOW(&g2);
  SHOW(&p);
}
int main() {
  SHOW(f);
  f(1);
}
```

```
f = 0x492d51a9
&g1 = 0x492d87e0
&g2 = 0x492d8820
&p = 0xa06f0020
```



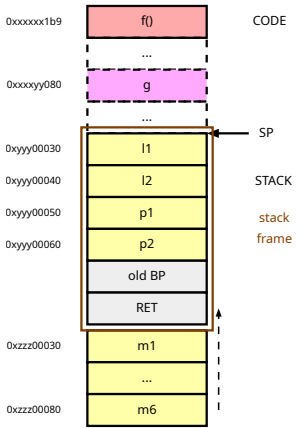
Stack-Segment (i)

- Funktionsparameter und lokale Variablen liegen im Stack-Segment

```c
int g = 1;
void f(int p1, int p2) {
  int l1, l2;
  SHOW2(&l1, &l2);
  SHOW2(&p1, &p2);
}
int main() {
  SHOW(&f);
  SHOW(&g);
  int m1, m2, m3, m4, m5, m6;
  f(1, 2);
  SHOW2(&m1, &m2);
  SHOW2(&m3, &m4);
  SHOW2(&m5, &m6);
}
```

- Parameter und lokale Variablen eines Funktionsaufrufs bilden ein Stack-Frame am Stack

```
&f = 0x10e111c9
&g = 0x10e14080
&l1 = 0x66bf0030;  &l2 = 0x66bf0040
&p1 = 0x66bf0050;  &p2 = 0x66bf0060
&m1 = 0x66cf0030;  &m2 = 0x66cf0040
&m3 = 0x66cf0050;  &m4 = 0x66cf0060
&m5 = 0x66cf0070;  &m6 = 0x66cf0080
```



Stack-Segment (ii)

- Aufrufkonventionen (*calling conventions*) definieren
  - wo Parameter abgelegt werden, z.B. am Stack
  - wo Rückgabewert abgelegt wird, z.B. im Register `AX`
  - wer welche Register wiederherstellen muss
- gemeinsam mit weiteren Parametern bilden sie Application Binary Interface (ABI)
  - beschreibt Interoperabilität zur Laufzeit
  - erlaubt Interoperabilität zwischen Prozeduren
    - aus unterschiedlichen Programmiersprachen
    - von verschiedenen Compiler
- im Gegensatz dazu Application Programming Interface (API)
  - beschreibt Interoperabilität zur Compile-Zeit

Stack-Segment (iii)

- lokale Variablen nur in Funktion gültig
  - neuer Aufrufe → gleiches Stack-Frame[1]

```c
int f() {
  int x;
  SHOW(x);
  x = 2;
  SHOW(x);
  SHOW(&x);
}
int main() {
  f(); f();
}
```

```
x = 0
x = 2
&x = 0xd871c8e4
x = 2
x = 2
&x = 0xd871c8e4
```

- Rückgabe von Zeiger auf Stack
  - nach Funktionsaufruf ungültig
  - GCC ersetzt Rückgabewert durch `0`

```c
int* make_array() {
    int a[10];
    SHOW(a);
    return a; // NOTE: compiler warning
}
int main() {
    int* a1 = make_array();
    int* a2 = make_array();
    SHOW2(a1, a2);
}
```

```
a = 0x397b6b90
a = 0x397b6b90
a1 = 0x0;  a2 = 0x0
```

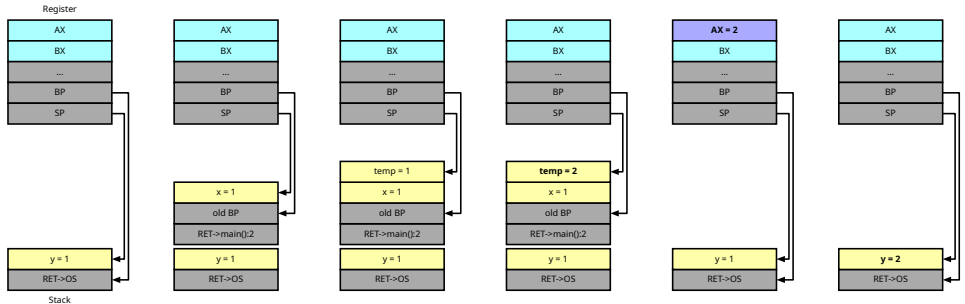
[1] in GCC verhindert, Beispiele übersetzt mit `-fno-stack-clash-protection`

Parameterübergabe

- Parameter über Stack übergeben
- Rückgabewert über Register AX
- Stackframe nur kurz gültig

```c
int inc(int x) {
    int temp = x;
    temp++;
    return temp;
}
```

```c
int main() {
    int y = 1;
    y = inc(y);
    SHOW(y);
}
```



Rekursion

- jeder rekursive Funktionsaufruf benötigt weiteres Stack-Frame
  - Rekursionstiefe daher in der Praxis eingeschränkt

```c
void count_recursive(size_t n) {
  fprintf(stderr, "n = %zu\n", n);
  count_recursive(n+1); // NOTE: infinite recursion (warning)
}
int main() {
  count_recursive(0);
}
```

```
n = 0
n = 1
n = 2
...
n = 312763
n = 312764
n = 312765
Job 1, './count_recursive' terminated by signal SIGSEGV (Address boundary error)
```

Heap-Segment und manuelle Speicherverwaltung (i)

- Speicher im Heap-Segment wird zur Laufzeit manuell (dynamisch) verwaltet
- Funktionen im Header `<stdlib.h>`
  - `malloc()` (*memory allocate*) Speicher reservieren
    - returniert `nullptr` falls nicht möglich
  - `free()` Speicher frei gegeben
- Verwendung wie normales Array, z.B.
  - für größere Datenmengen
  - Rückgabewerte von Prozeduren
- manuelle Speicherverwaltung ist historisch größte Hürde für C-Programmierung

```
int* a = malloc(sizeof(int)*10);
if (a != nullptr) {
  SHOW(a[0]); // NOTE: content not initialized
  a[0] = 123;
  SHOW(a[0]);
  free(a);
}

SHOW(a); // address of dynamic memory
SHOW(&a); // address of variable on stack
```

```
a[0] = -1094795586
a[0] = 123
a = 0x2bfe0010
&a = 0x2aaf0020
```

Heap-Segment und manuelle Speicherverwaltung (ii)

- Beispiel Rückgabe von Array
  - `calloc()` initialisiert Speicher mit `0`
  - manuell reservierter Speicher wird mit `free()` wieder freigegeben
    - ansonsten → Speicherleck (memory leak)

```
int* make_int_array(size_t n) {
    int* result = calloc(n, sizeof(int)); // <---- allocate ---.
    return result;                        //                   |
}                                         //                   |
int main() {                              //                   |
    int* a = make_int_array(10);          //                   |
    SHOW(a[0]);                           //                   +
    SHOW(a);                              //                  /|
    free(a); // <---- free ----------------------------------'  |
    a = make_int_array(20); //                                  |
    SHOW(a);                //                                  |
    free(a); // <---- free ----------------------------------'
}
```

```
a[0] = 0
a = 0x25e0010
a = 0x25e0020
```

Faustregel: *Für jeden Aufruf von `malloc()` oder `calloc()` muss es einen entsprechenden Aufruf von `free()` geben.*

Heap-Segment und manuelle Speicherverwaltung (iii)

Beispiel: Verarbeitung von großen Datenmengen nur am Heap möglich

- Speicher am Stack (3 MiB)
  - Programm bricht zur Laufzeit ab

```
constexpr size_t MAX = 3 * 1024 * 1024;
void huge_stack() {
    int a[MAX] = {};
    SHOW(a[MAX-1]);
}
int main() {
    huge_stack();
}
```

```
stack-overflow on address 0x7fff5efbf300
    #0 0x5596cca641c2 in huge_stack
    #1 0x5596cca642e4 in main
    #2 0x7f9be0e27674 libc
    #3 0x7f9be0e27728 in __libc_start_main
    #4 0x5596cca640d4 in _start
```

- Speicher auf Heap (1024 MiB)
  - kein Problem

```
constexpr size_t MAX = 1024 * 1024 * 1024;

void huge_heap() {
    int* a = calloc(MAX, sizeof(int));
    SHOW(a[MAX-1]);
    free(a);
}

int main() {
    huge_heap();
}
```

```
a[MAX-1] = 0
```

Sanitizer & Memory Profiler

- Sanitizer erzeugen zusätzlichen Code der zur Laufzeit die “Vernünftigkeit” (*sanity*) des Programms überprüfen
  - Address Sanitizer (ASan)
    - use after free/return/scope
    - heap/stack/global buffer overflow
    - memory leaks
    - reduziert Geschwindigkeit um ca. 50%
  - Memory Sanitizer (MSan)
    - uninitilized memory access
    - reduziert Geschwindigkeit um ca. 75%
  - Undefined Behavior Sanitizer (UBSan)
    - erkennt undefiniertes Verhalten ~40 Prüfungen
    - kaum Laufzeitoverhead
  - können durch Compiler-Parameter aktiviert werden,
    - z.B. `gcc -fsanitize=address,undefined ...`

- Speicher-Profiler, wie, z.B. Valgrind Memcheck
  - exaktes Verfolgen von gültigem Speicherzugriffen
  - Rückschlüsse auf Speicherlecks
  - langsamer aber gründlicher als Sanitizer
  - z.B.

```
gcc -o hello hello.c
valgrind --leak-check=yes ./hello
```

```
HEAP SUMMARY:
    in use at exit: 0 bytes in 0 blocks
  total heap usage: 1 allocs, 1 frees,
                    4,096 bytes allocated

All heap blocks were freed
    -- no leaks are possible
```